

1.2 Input & Variables

Sue is excited because she just got a list of ancestor names from her grandmother. Finally, she can get some traction on her genealogy work! Unfortunately, the names are in the wrong order. Rather than being in the format of [LastName, FirstName MiddleInitial], they are [FirstName MiddleInitial LastName]. Instead of retyping the entire list, Sue writes a program to swap the names.

Objectives

By the end of this class, you will be able to:

- Choose the best data-type to represent your data.
- Declare a variable.
- Accept user input from the keyboard and store it in a variable.

Prerequisites

Before reading this section, please make sure you are able to:

- Type the code for a simple program (Chapter 0.2).
- Use the provided tools to complete a homework assignment (Chapter 1.0).
- Display text and numbers on the screen (Chapter 1.1).

Overview

Variables in computer languages are much like variables in mathematics:

Variables are a named location where we store data

There are two parts to this definition. The first part is the name. We always refer to variables by a name which the programmer identifies. It is always worthwhile to make the name as unambiguous as possible so it won't get confused with other variables or used later in the program. The second part is the data. A wide variety of data-types can be stored in a variable.

Variables

All the data in a computer is stored in memory. This memory consists of collections of 1's and 0's which are meant to represent numbers, letters, and text. There are two main considerations when working with variables: how to interpret the memory into something (like the number 3.8 or the text "Computer Science"), and what that something means (like your GPA or your major).

There is no intrinsic meaning for these 1's and 0's; they could mean or refer to just about anything. It is therefore the responsibility of the programmer to specify how to interpret these 1's and 0's. This is done through the data-type. A data-type can be thought of as a formula through which the program interprets the 1's and 0's in memory. An integer number, for example, is interpreted quite differently than a real number or a letter. Every computer has a built-in set of data-types facilitating working with text, numbers, and logical data. C++ facilitates these built-in data-types with the following type names:

Data-type	Use	Size	Range of values
bool	Logic	1	true, false
char	Letters and symbols	1	-128 to 127 ... or 'a', 'b', etc.
short	Small numbers, Unicode characters	2	-32,767 to 32,767
int	Counting	4	-2 billion to 2 billion
long (long int)	Larger Numbers	8	$\pm 9,223,372,036,854,775,808$
float	Numbers with decimals	4	10^{-38} to 10^{38} accurate to 7 digits
double	Larger numbers with decimals	8	10^{-308} to 10^{308} accurate to 15 digits
long double	Huge Numbers	16	10^{-4932} to 10^{4932} accurate to 19 digits

Thus when you declare a variable to be an integer (int), the 1's and 0's in memory will be interpreted using the integer formula and only integer data can be stored in the variable.

Sam's Corner

Under the covers, all data in a computer is represented as a charge stored in a very small capacitor on a chip. We call these bits, 1 indicating “true” (corresponding to a charge in the capacitor) and 0 corresponding to “false” (corresponding to no charge). Bits are stored differently on CDs, flash memory, and hard drives.



Integers

Integers are possibly the most commonly used data-type. Integers are useful for counting or for numbers that cannot have a decimal. For example, the number of members in a family is always an integer; there can never be 2.4 people in a family. You can declare a variable as an integer with:

```
int age = 42;
```

With this line of code, a new variable is created. The name is “age” which is how the variable will be referenced for the remainder of the program. Since the data-type is an integer (as specified by the int keyword), we know two things. First, the amount of memory used by the variable is 4 bytes (1 byte equals 8 bits so it takes a total of 32 bits to store one integer). Second, the value of the variable age must be between -2,147,483,648 and 2,147,483,647. Observe how the integer is initialized to the value of 42 in this example.

Sam's Corner

It is easiest to explain how integers are stored in memory by considering a special integer that is only positive and has 8 bits (this is actually an unsigned char). In this case, the right-most bit correspond to the 1's place, the next corresponds to the 2's place, the next corresponds to the 4's place, and so on. Thus the bits (00101010) is interpreted as:

$$\begin{array}{cccccccc}
 2^7 & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\
 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\
 \hline
 0 & + & 0 & + & 32 & + & 0 & + & 8 & + & 0 & + & 2 & + & 0 & = & 42
 \end{array}$$

In other words, each place has a value corresponding to it (as a power of two because we are counting in binary). You add that value to the sum only if there is a 1 in that place. Typically integers are 32 bits (4 bytes) in length. The left-most bit is special, indicating whether the number is positive or negative.



Floating point numbers

In mathematics, real numbers are numbers that can have a decimal. It is often convenient to represent very large or very small real numbers in scientific notation:

$$1888 = 1.888 \times 10^3$$

Observe how the decimal point position is specified by the exponent (10^3 in this case). In many ways, the decimal point can be said to “float” or move according to the exponent, the origin of the term “floating point numbers” in computer science. Floating point numbers are characterized by two parts: the precision part of the equation (1.888 in the above example) and the exponent (10^3). There are three floating point types available in the C++ language:

Type name	Memory used	Exponent	Precision
float	4	10^{-38} to 10^{38}	7 digits
double	8	10^{-308} to 10^{308}	15 digits
long double	16	10^{-4932} to 10^{4932}	19 digits

Observe how the more data is used (measured in bytes), the more accurately the number can be represented. However, all floating point numbers are approximations. Examples of declaring floating point numbers include:

```
float gpa = 3.9;
double income = 103295.05;
long double pi = 3.14159265358979323;
```



Sue's Tips

While it is wasteful to use a larger data-type than is strictly necessary (who would ever want their GPA to be represented to 19 digits?), it is much worse to not have sufficient room to store a number. In other words, it is a good idea to leave a little room for growth when declaring a floating point number.

Characters

Another common data-type is a character, corresponding to a single letter, number, or symbol. We declare a character variable with:

```
char grade = 'A';
```

When making an assignment with chars, a single ' is used on each side of the character. This is different than the double quotes " used when denoting text. Each character in the char range has a number associated with it. This mapping of numbers to characters is called the ASCII table:

Number	46	47	48	49	50	...	65	66	67	68	...	97	98	99	100
Letter	.	/	0	1	2		A	B	C	D		a	b	c	d

The complete ASCII table can be viewed in a variety of web sites:

<http://en.cppreference.com/w/cpp/language/ascii>

Sam's Corner



The char data-type is actually a form of integer where the size is 1 byte (or 8 bits). This means there are only 256 possible values, not four billion. Each number in the char range corresponds to a glyph (letter, number, or symbol) in the ASCII table. Thus you can treat a char like a letter or you can do math with it like any other integer. For example, 'A' + 1 is the same as 'B', which is 66.

Text

Text consists of a collection or string of characters. While all the data-types listed below can readily fit into a small slot in memory, text can be exceedingly long. For example, the amount of memory necessary to store your name is much less than that required to store a complete book. You declare a string variable with:

```
char text[256] = "CS 124";
```

There are a few things to observe about this declaration. First, the size of the buffer (or number of available slots in the string) is represented in square brackets []. The programmer specifies this size at compile time and it cannot be changed. The second thing to note is how the contents of the string are surrounded in double quotes " just as they were with our cout examples.



Sue's Tips

The standard size to make strings is 256 characters in length. This is plenty long enough for most applications. It is usually more convenient (and bug-free) to have the same string length for an entire project than to have many different string buffer sizes (which would require us to keep track of them all!).

Logical Data

The final built-in data-type is a bool. This enables us to capture data having only two possible values. For example, a person is either pregnant or not, either alive or not, either male or not, or either a member of the church or not. For these data-types, we use a bool:

```
bool isMale = false;
```

There are only two possible values for a bool: true or false. By convention, we name bool variables in such a way that we know what true means. In other words, it would be much less helpful to have a variable called gender. What does false mean (that one *has* no gender like a rock)?

Sam's Corner



A bool takes a single byte of memory, consisting of 8 bits. Note that we really only need a single bit to capture Boolean (true/false) data. Why do we need 8 then? This has to do with how convenient it is for the computer to work with bytes and how awkward it is to work with bits. When evaluating a bool, any 1's in any of the bits will result in a true evaluation. Only when all 8 bits are 0 will the bool evaluate to false. This means that there are 255 true values ($2^8 - 1$) and 1 false value.

Input

Now that we know how to store data in a computer program using variables, it is possible to prompt the user for input. Note that without variables we would not have a place to store the user input so asking the user questions would be futile. The main mechanism with which we prompt users for input is the `cin` function. This function, like `cout`, is part of the `iostream` library. The code for prompting the user for his age is:

```
{
    int age;
    cin >> age;
}
```

In this example, we first declare a variable that can hold an integer. There are a couple important points here:

- Use `cin` rather than `cout`. This refers to Console INput, analogous to the Console OUtput of `cout`.
- The **extraction operator** `>>` is used instead of the **insertion operator** `<<`. Again, the arrow points the direction the data goes. In this case, it goes from the keyboard (represented by `cin`) to the variable (represented by `age`).
- There is always a variable on the right side of the extraction operator.

We can use `cin` with all built in data-types:

```
{
    // INTEGERS
    int age;           // integers can only hold digits.
    cin >> age;        // if a non-digit is entered, then age remains uninitialized.

    // FLOATS
    double price;      // able to handle zero or many digits
    cin >> price;

    // SINGLE LETTERS
    char letter;        // only one letter of any kind
    cin >> letter;      // anything but a white-space (space, tab, or newline).
    cin.get(letter);    // same as above, but will also get white-spaces

    // TEXT
    char name[256];     // any text up to 255 characters in length
    cin >> name;        // all user input up to the first white-space is accepted
}
```



A stream (the input from the keyboard into `cin`) can be thought of as a long list of characters moving from the keyboard into your program. The question is: how much input is consumed by a single `cin` statement? Consider the following input stream:

4	2	C	e	l	s	i	u	s	
---	---	---	---	---	---	---	---	---	--

And consider the code:

```
int temperature;
char units[256];
cin >> temperature;
cin >> units;
```

In this example, the input stream starts at the space before the 4. The first thing that happens is that all the white-spaces are skipped. This moves the cursor to the 4. Since a 4 is a digit, it can be put into the integer `temperature`. Thus the value in `temperature` is 4 and the cursor advances to the next spot. From here, 2 is recognized as a digit so the 4 value in `temperature` becomes 40 and 2 is added to yield 42. Again the cursor is advanced. At this point, `c` is not a digit so we stop accepting input in the variable `temperature`. The next `cin` statement is executed which accepts text. Recall that text accepts input up to the first white-space. Since the cursor is on the `c`, the entire word of “Celsius” will be put in the `units` variable and the cursor will stop at the white-space.

Multiple Extraction Operators

Often it is convenient to input data into more than one variable on a single input statement. This can be done by “stacking” the extraction operators much like we stacked the insertion operators:

```
{
    char name[256];
    int age;
    cin >> name >> age;
}
```

In this example, the first thing the user inputs will be put into the `name` variable and the second into `age`.

Whole Lines of Text

Recall how, when reading text into a variable using `cin`, only one word (or more accurately the characters between white-spaces) are entered. What do you do when you want to enter an entire line of text including the spaces? For this scenario, a new mechanism is needed:

```
{
    char fullName[256];           // store an individual's full name: Dr. Drake Ramoray
    cin.getline(fullName, 256);
}
```

Observe how we do not use the extraction (`>>`) operator which was part of our other input mechanisms. The `getline` function takes two parameters: the name of the variable (`fullName` in this example) and the length of the buffer (256 because that is how large `fullName` was when it was defined).

Example 1.2 – Many Prompts

Demo	This example will demonstrate how to declare text, integer, floating point, and character variables. It will also demonstrate how to accept data from the user with each of these data types.
Problem	Write a program to prompt the user for his first name, age, GPA, and the expected grade in CS 124. The information will then be displayed on the screen. <pre>What is your first name: Sam What is your age: 19 What is your GPA: 3.91 What grade do you hope to get in CS 124: A Sam, you are 19 with a 3.9 GPA. You will get an A.</pre>
Solution	The four variables are declared as follows: <pre>char name[256]; int age; float gpa; char letterGrade;</pre> To prompt the user for his age, it is necessary to display a prompt first so the user knows what to do. Usually we precede the prompt and the input with a comment and blank line: <pre>// Prompt the user for his age cout << "What is your age: "; cin >> age;</pre> Finally, we must not forget to format cout to display one digit after the decimal. <pre>// configure the display to show GPAs: one digit of accuracy cout.setf(ios::fixed); cout.setf(ios::showpoint); cout.precision(1); // display the results cout << "\t" << name << ", you are " << age << " with a " << gpa << " GPA. You will get an " << letterGrade << ".\n";</pre>
Challenge	As a challenge, try to accept an individual's full name (Such as "Sam S. Student") rather than just the first name. Also, try to configure the output to display two digits of accuracy rather than one.
See Also	The complete solution is available at 1-2-manyPrompts.cpp or: <pre>/home/cs124/examples/1-2-manyPrompts.cpp</pre> 

Problem 1

What is the output of the following line of code?

```
cout << "\\\"/\n";
```

Answer:

Please see page 29 for a hint.

Problem 2

How do you put a tab on the screen?

Answer:

Please see page 27 for a hint.

Problem 3

How do you output the following:

```
You will need to use '\n' a ton in this class.
```

Answer:

Please see page 29 for a hint.

Problem 4

How do you declare an integer variable?

Answer:

Please see page 36 for a hint.

Problem 5

How would you declare a variable for each of the following?

Variable name	Declaration
yearBorn	
gpa	
nameStudent	
ageStudent	

Please see page 36 for a hint.

Problem 6

Declare a variable to store the ratio of feet to meters.

Answer:

Please see page 37 for a hint.

Problem 7

What is the number of bytes for each data type?

```
{  
    cout << sizeof(char) << endl;  
  
    char a;  
    cout << sizeof(a) << endl;  
  
    cout << sizeof(bool) << endl;  
  
    int b;  
    cout << sizeof(b) << endl;  
  
    float c;  
    cout << sizeof(c) << endl;  
  
    double d;  
    cout << sizeof(d) << endl;  
  
    long double e;  
    cout << sizeof(e) << endl;  
}
```

Please see page 36 for a hint.

Problem 8

Which of the following can store the largest number?

```
bool value;  
char value[256];  
int value  
long double value;
```

Please see page 36 for a hint.

Problem 9

Declare a variable to represent the following number in C++: 8,820,198,883,463.39

Answer:

Please see page 37 for a hint.

Problem 10

Write the code to prompt a person for his first name.

Answer:

Please see page 39 for a hint.

Problem 11

Write the code to prompt a person for his two favorite numbers.

Answer:

Please see page 39 for a hint.

Problem 12

Write the code to prompt a person for his full name.

Answer:

Please see page 40 for a hint.

Assignment 1.2

Write a program that prompts the user for his or her income and displays the result on the screen. There will be two parts:

Get Income

The first part is code that prompts the user for his income. It will ask the user:

```
Your monthly income:
```

There will be a tab before “Your” and a single space after the “:”. There is no newline at the end of this prompt. The user will then provide his or her income as a float.

Display

The second part is code to display the results to the screen.

```
Your income is: $ 1010.99
```

Note that there is one space between the colon and the dollar sign. The money is right aligned to 9 spaces from the dollar sign.

Example

User input is underlined. Note that you will not be making the input underlined; this is just the notation used in the assignments to distinguish input from output.

```
Your monthly income: 932.16
Your income is: $ 932.16
```

Instructions

Please verify your solution against:

```
testBed cs124/assign12 assignment12.cpp
```

Don’t forget to submit your assignment with the name “Assignment 12” in the header.

Please see page 41 for a hint.